

ЛАБОРАТОРНАЯ РАБОТА № 05.

«Введение в функциональное программирование и Stream API»

Группа: РИМ-150950

Студент: Ляхова Елизавета Олеговна

Ссылка на github: https://github.com/Lizaswer/java_course_master.git

Цель: получить представление о функциональном программировании и
Stream API

Ход работы:

1. Реализация всех примеров, которые даны в начале работы.

Example1EvenNumbers

```
package lrl5.examples;

import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;

public class Example1EvenNumbers {
    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);
        System.out.println("Введите размер массива");
        int size = in.nextInt();

        int[] arr = new int[size];
        Random random = new Random();

        for (int i = 0; i < size; i++) {
            arr[i] = random.nextInt(200);
        }

        System.out.println("Массив arr : ");
        System.out.println(Arrays.toString(arr));

        int[] arrResult = getEvenNumbers(arr);

        System.out.println("Массив arrResult (четные  
числа) : ");

        System.out.println(Arrays.toString(arrResult));
    }

    public static int[] getEvenNumbers(int[] arr) {
        return Arrays.stream(arr)
```

```
        .filter(n -> n % 2 == 0)
        .toArray();
    }
}
```

Example2FindCommonElements

```
package lrl5.examples;

import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;

public class Example2FindCommonElements {
    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);
        System.out.println("Введите размер массива");
        int size = in.nextInt();

        int[] arr1 = new int[size];
        int[] arr2 = new int[size];
        Random random = new Random();

        for (int i = 0; i < size; i++) {
            arr1[i] = random.nextInt(200);
            arr2[i] = random.nextInt(200);
        }

        System.out.println("Массив arr1 : ");
        System.out.println(Arrays.toString(arr1));
        System.out.println("Массив arr2 : ");
        System.out.println(Arrays.toString(arr2));

        int[] arrResult = findCommonElements(arr1,
arr2);

        System.out.println("Массив arrResult : ");
```

```

System.out.println(Arrays.toString(arrResult));
    }

    public static int[] findCommonElements(int[]
arr1, int[] arr2) {
        return Arrays.stream(arr1)
            .filter(x -> Arrays.stream(arr2)
                .anyMatch(y -> y == x))
            .distinct()
            .toArray();
    }
}

```

Example3CapitalizedString

```

package lrl5.examples;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Example3CapitalizedString {
    public static void main(String[] args) {
        String text = "Напишите функцию Которая
принимает на вход список Строки и возвращает новый
список Содержащий только те строки Которые
начинаются с большой буквы";
        List<String> strings =
Arrays.asList(text.split(" "));

        System.out.println("Исходный список строк:");
        for (String e : strings) {
            System.out.println(e);
        }

        List<String> result =
filterCapitalizedStrings(strings);
    }
}

```

```

        System.out.println("\nСтроки, начинающиеся с
большой буквы:");
        for (String e : result) {
            System.out.println(e);
        }
    }

    public static List<String>
filterCapitalizedStrings(List<String> list) {
        return list.stream()
            .filter(s -> !s.isEmpty() &&
Character.isUpperCase(s.charAt(0)))
            .collect(Collectors.toList());
    }
}

```

Example4Squares

```

package lrl5.examples;

import java.util.List;
import java.util.stream.Collectors;
import java.util.ArrayList;
import java.util.Random;

public class Example4Squares {

    public static void main(String[] args) {
        List<Integer> integers = new ArrayList<>();
        Random random = new Random();

        System.out.println("\n" + "Список: " + "\n");

        for(int i = 0; i < 10; i++) {
            integers.add(random.nextInt(10000));
            System.out.println(integers.get(i));
        }
    }
}

```

```

        List<Integer> interperAfter =
squaresList(integers);

        System.out.println("\n" + "Список после
возведения в квадрат: " + "\n");

        for (Integer i : interperAfter) {
            System.out.println(i);
        }
    }

    private static List<Integer>
squaresList(List<Integer> list) {
        return list.stream().map(x -> x *
x).collect(Collectors.toList());
    }
}

```

2. Напишите функцию, которая принимает на вход массив целых чисел и возвращает новый массив, содержащий только четные числа из исходного массива.

```

package lrl5.tasks;

import java.util.Arrays;

public class Task2EvenNumbers {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9,
10};

        int[] evens = getEvenNumbers(numbers);
        System.out.println("Исходный массив: " +
Arrays.toString(numbers));
        System.out.println("Четные числа: " +
Arrays.toString(evens));
    }
}

```

```

    public static int[] getEvenNumbers(int[] arr) {
        return Arrays.stream(arr)
            .filter(n -> n % 2 == 0)
            .toArray();
    }
}

```

3. Напишите функцию, которая принимает на вход два массива целых чисел и возвращает новый массив, содержащий только те элементы, которые есть в обоих массивах.

```

package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task4Intersection {
    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3, 4, 5, 6};
        int[] arr2 = {4, 5, 6, 7, 8, 9};

        int[] result = getIntersection(arr1, arr2);
        System.out.println("Общие элементы: " +
Arrays.toString(result));
    }

    public static int[] getIntersection(int[] arr1,
int[] arr2) {
        Set<Integer> set1 =
Arrays.stream(arr1).boxed().collect(Collectors.toSet(
));

        return Arrays.stream(arr2)
            .filter(set1::contains)
            .distinct()
            .toArray();
    }
}

```

4. Напишите функцию, которая принимает на вход список строк и возвращает новый список, содержащий только те строки, которые начинаются с большой буквы.

```
package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task3CapitalizedStrings {
    public static void main(String[] args) {
        List<String> words = Arrays.asList("Java",
"python", "C++", "javascript", "Kotlin");
        List<String> result =
getCapitalizedStrings(words);
        System.out.println("С большой буквы: " +
result);
    }

    public static List<String>
getCapitalizedStrings(List<String> list) {
        return list.stream()
            .filter(s -> !s.isEmpty() &&
Character.isUpperCase(s.charAt(0)))
            .collect(Collectors.toList());
    }
}
```

5. Напишите функцию, которая принимает на вход список чисел и возвращает новый список, содержащий квадраты этих чисел.

```
package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;
```



```

public class Task5Squares {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3,
4, 5);
        List<Integer> squares = getSquares(numbers);
        System.out.println("Квадраты: " + squares);
    }

    public static List<Integer>
getSquares(List<Integer> list) {
        return list.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());
    }
}

```

6. Напишите функцию, которая принимает на вход список строк и возвращает новый список, содержащий только те строки, которые содержат заданную подстроку.

```

package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task6ContainsSubstring {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("apple",
"banana", "grape", "apricot", "cherry");
        String substr = "ap";
        List<String> result =
filterBySubstring(strings, substr);
        System.out.println("Содержат '" + substr +
"': " + result);
    }
}

```

```

    public static List<String>
filterBySubstring(List<String> list, String
substring) {
    return list.stream()
        .filter(s -> s.contains(substring))
        .collect(Collectors.toList());
}
}

```

7. Напишите функцию, которая принимает на вход список целых чисел и возвращает новый список, содержащий только те числа, которые делятся на заданное число без остатка.

```

package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task7DivisibleBy {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(10, 15,
20, 25, 30, 35);
        int divisor = 5;
        List<Integer> result = getDivisibleBy(numbers,
divisor);
        System.out.println("Делятся на " + divisor +
": " + result);
    }

    public static List<Integer>
getDivisibleBy(List<Integer> list, int divisor) {
        return list.stream()
            .filter(n -> n % divisor == 0)
            .collect(Collectors.toList());
    }
}

```

8. Напишите функцию, которая принимает на вход список строк и

возвращает новый список, содержащий только те строки, которые имеют длину больше заданного значения.

```
package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task8LongerThan {
    public static void main(String[] args) {
        List<String> words = Arrays.asList("cat",
"elephant", "dog", "butterfly", "ant");
        int minLength = 4;
        List<String> result = getLongerThan(words,
minLength);
        System.out.println("Длиннее " + minLength +
": " + result);
    }

    public static List<String>
getLongerThan(List<String> list, int minLength) {
        return list.stream()
            .filter(s -> s.length() > minLength)
            .collect(Collectors.toList());
    }
}
```

9. Напишите функцию, которая принимает на вход список целых чисел и возвращает новый список, содержащий только те числа, которые больше заданного значения.

```
package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task9GreaterThan {
    public static void main(String[] args) {
```

```

        List<Integer> numbers = Arrays.asList(5, 12,
8, 20, 3, 15);
        int threshold = 10;
        List<Integer> result = getGreaterThan(numbers,
threshold);
        System.out.println("Больше " + threshold + ":
" + result);
    }

    public static List<Integer>
getGreaterThan(List<Integer> list, int threshold) {
        return list.stream()
            .filter(n -> n > threshold)
            .collect(Collectors.toList());
    }
}

```

10. Напишите функцию, которая принимает на вход список строк и возвращает новый список, содержащий только те строки, которые содержат только буквы (без цифр и символов).

```

package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task10OnlyLetters {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("hello",
"world123", "Java", "C++", "Lambda");
        List<String> result = getOnlyLetters(strings);
        System.out.println("Только буквы: " +
result);
    }
}

```

```

    public static List<String>
getOnlyLetters(List<String> list) {
    return list.stream()
        .filter(s ->
s.matches("[A-Za-zА-Яа-я]+"))
        .collect(Collectors.toList());
    }
}

```

11. Напишите функцию, которая принимает на вход список целых чисел и возвращает новый список, содержащий только те числа, которые меньше заданного значения.

```

package lrl5.tasks;

import java.util.*;
import java.util.stream.Collectors;

public class Task11LessThan {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(5, 12,
8, 20, 3, 15);
        int threshold = 10;
        List<Integer> result = getLessThan(numbers,
threshold);
        System.out.println("Меньше " + threshold + ":
" + result);
    }

    public static List<Integer>
getLessThan(List<Integer> list, int threshold) {
        return list.stream()
            .filter(n -> n < threshold)
            .collect(Collectors.toList());
    }
}

```

12. Решите две любых задачи с сайта timus.

Задача 1313.

```
package lrl5.timus;

import java.util.Scanner;

public class Task1313 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        int[][] a = new int[N][N];

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                a[i][j] = sc.nextInt();
            }
        }

        for (int diag = 0; diag <= 2 * (N - 1);
diag++) {
            // Проходим по всем строкам снизу вверх
            for (int i = N - 1; i >= 0; i--) {
                int j = diag - i;
                if (j >= 0 && j < N) {
                    System.out.print(a[i][j] + " ");
                }
            }
        }

        sc.close();
    }
}
```

Задача 1327.

```
package lrl5.timus;

import java.util.Scanner;

public class Task1327 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int A = sc.nextInt();
        int B = sc.nextInt();

        sc.close();

        int count = 0;

        for (int day = A; day <= B; day++) {
            if (day % 2 == 1) { // нечетный день
                count++;
            }
        }

        System.out.println(count);
    }
}
```

Вывод:

В результате выполнения лабораторной работы я изучил основные возможности Stream API в Java: методы `filter()`, `map()`, `collect()`, `toArray()`, а также лямбда-выражения. Были реализованы 11 функций для обработки массивов и списков, а также решены две задачи с сайта Timus. Функциональное программирование позволяет писать более лаконичный и понятный код по сравнению с императивным подходом.

